

# brilliant-cv

## Documentation

|                                                                                                  |
|--------------------------------------------------------------------------------------------------|
| <p><i>Authors: yunanwg</i></p> <p><i>Build Date: 2025-07-19</i></p> <p><i>Version: 2.0.5</i></p> |
|--------------------------------------------------------------------------------------------------|

### Contents

|                                          |   |
|------------------------------------------|---|
| 1. Introduction .....                    | 2 |
| 2. Setup .....                           | 2 |
| 3. Migration from v1 to v2 .....         | 3 |
| 4. Configuration via metadata.toml ..... | 4 |
| 5. Functions .....                       | 6 |

## 1. Introduction

Brilliant CV is a Typst template for making Résumé, CV or Cover Letter inspired by the famous LaTeX CV template Awesome-CV.

## 2. Setup

### Step 1: Install Fonts

In order to make Typst render correctly, you will have to install the required fonts [Roboto](#) and [Source Sans Pro](#) (or Source Sans 3) in your local system.

### Step 2: Check Documentation

You are reading this documentation now, woah!

### Step 3: Bootstrap Template

In your local system, just working like `git clone`, bootstrap the template using this command:

```
1 typst init @preview/brilliant-cv:<version>
```

bash

Replace the `<version>` with the latest or any releases (after 2.0.0).

### Step 4: Compile Files

Adapt the `metadata.toml` to suit your needs, then `typst c cv.typ` to get your first CV!

### Step 5: Go beyond

It is recommended to:

1. Use `git` to manage your project, as it helps trace your changes and version control your CV.
2. Use `typstyle` and `pre-commit` to help you format your CV.
3. Use `typos` to check typos in your CV if your main locale is English.
4. (Advanced) Use `LTEX` in your favorite code editor to check grammars and get language suggestions.

### 3. Migration from v1 to v2

With an existing CV project using the v1 version of the template, a migration is needed, including replacing some files / some content in certain files.

1. Delete brilliant-CV folder, .gitmodules. (Future package management will directly be managed by Typst)
2. Migrate all the config on metadata.typ by creating a new metadata.toml. Follow the example toml file in the repo,

it is rather straightforward to migrate.

3. For cv.typ and letter.typ, copy the new files from the repo, and adapt the modules you have in your project.

4. For the module files in /modules\_\* folders:

- a. Delete the old `import #import "../brilliant-CV/template.typ": *`, and replace it by the import statements in the new template files.

- b. Due to the Typst path handling mechanism, one cannot directly pass the path string to some functions anymore. This concerns, for example, the logo argument in cvEntry, but also on cvPublication as well. Some parameter names were changed, but most importantly, you should pass a function instead of a string (i.e. `image("logo.png")` instead of `"logo.png"`). Refer to new template files for reference.

- c. You might need to install Roboto and Source Sans Pro on your local system now, as new Typst package discourages including these large files.

- d. Run `typst c cv.typ` without passing the font-path flag. All should be good now, congrats!

Feel free to raise an issue for more assistance should you encounter a problem that you cannot solve on your own :)

## 4. Configuration via metadata.toml

The metadata.toml file is the main configuration file for your CV. By changing the key-value pairs in the config file, you can setup the names, contact information, and other details that will be displayed in your CV.

Here is an example of a metadata.toml file:

```
1  # INFO: value must matches folder suffix; i.e "zh" -> "./modules_zh"
2  language = "en"
3
4  [layout]
5  # Optional values: skyblue, red, nephritis, concrete, darknight
6  awesome_color = "skyblue"
7
8  # Skips are for controlling the spacing between sections and entries
9  before_section_skip = "1pt"
10 before_entry_skip = "1pt"
11 before_entry_description_skip = "1pt"
12
13 [layout.header]
14 # Optional values: left, center, right
15 header_align = "left"
16
17 # Decide if you want to display profile photo or not
18 display_profile_photo = true
19 profile_photo_path = "template/src/avatar.png"
20
21 [layout.entry]
22 # Decide if you want to put your company in bold or your position in bold
23 display_entry_society_first = true
24
25 # Decide if you want to display organisation logo or not
26 display_logo = true
27
28 [inject]
29 # Decide if you want to inject AI prompt or not
30 inject_ai_prompt = false
31
32 # Decide if you want to inject keywords or not
33 inject_keywords = true
34 injected_keywords_list = ["Data Analyst", "GCP", "Python", "SQL", "Tableau"]
35
36 [personal]
37 first_name = "John"
38 last_name = "Doe"
39
40 # The order of this section will affect how the entries are displayed
```

```

41 # The custom value is for any additional information you want to add
42 [personal.info]
43 github = "yunanwg"
44 phone = "+33 6 12 34 56 78"
45 email = "john.doe@me.org"
46 linkedin = "johndoe"
47 # gitlab = "yunanwg"
48 # homepage: "jd.me.org"
49 # orcid = "0000-0000-0000-0000"
50 # researchgate = "John-Doe"
51 # extraInfo = "I am a cool kid"
52 # custom-1 = (icon: "", text: "example", link: "https://example.com")
53
54 # add a new section if you want to include the language of your choice
55 # i.e. [[lang.ru]]
56 # each section must contains the following fields
57 [lang.en]
58 header_quote = "Experienced Data Analyst looking for a full time job starting
59 from now"
60 cv_footer = "Curriculum vitae"
61 letter_footer = "Cover letter"
62
63 [lang.fr]
64 header_quote = "Analyste de données expérimenté à la recherche d'un emploi à
65 temps plein disponible dès maintenant"
66 cv_footer = "Résumé"
67 letter_footer = "Lettre de motivation"
68
69 [lang.zh]
70 header_quote = "oooooooooooooooooooo"
71 cv_footer = ""
72 letter_footer = ""
73
74 # For languages that are not written in Latin script
75 # Currently supported non-latin language codes: ("zh", "ja", "ko", "ru")
76 [lang.non_latin]
77 name = ""
78 font = "Heiti SC"

```

## 5. Functions

### cvEntry

Add an entry to the CV.

- title (str): The title of the entry.
- society (str): The society of the entry (company, university, etc.).
- date (str | content): The date(s) of the entry.
- location (str): The location of the entry.
- description (array): The description of the entry. It can be a string or an array of strings.
- logo (image): The logo of the society. If empty, no logo will be displayed.
- tags (array): The tags of the entry.
- metadata (array): (optional) the metadata read from the TOML file.
- awesomeColors (array): (optional) the awesome colors of the CV.

### Parameters

```
cvEntry(  
    title,  
    society,  
    date,  
    location,  
    description,  
    logo,  
    tags,  
    metadata,  
    awesomeColors  
) -> content
```

### cvEntryContinued

Add a continued entry to the CV.

- title (str): The title of the entry.
- date (str | content): The date(s) of the entry.
- description (array): The description of the entry. It can be a string or an array of strings.
- tags (array): The tags of the entry.
- metadata (array): (optional) the metadata read from the TOML file.
- awesomeColors (array): (optional) the awesome colors of the CV.

### Parameters

```
cvEntryContinued(  
    title,  
    date,  
    description,  
    tags,  
    metadata,  
    awesomeColors  
) -> content
```

### **cvEntryStart**

Add the start of an entry to the CV.

- society (str): The society of the entry (company, university, etc.).
- location (str): The location of the entry.
- logo (image): The logo of the society. If empty, no logo will be displayed.
- metadata (array): (optional) the metadata read from the TOML file.
- awesomeColors (array): (optional) the awesome colors of the CV.

#### **Parameters**

```
cvEntryStart(  
  society,  
  location,  
  logo,  
  metadata,  
  awesomeColors  
) -> content
```

### **cvHonor**

Add a Honor to the CV.

- date (str): The date of the honor.
- title (str): The title of the honor.
- issuer (str): The issuer of the honor.
- url (str): The URL of the honor.
- location (str): The location of the honor.
- awesomeColors (array): (optional) The awesome colors of the CV.
- metadata (array): (optional) The metadata read from the TOML file.

#### **Parameters**

```
cvHonor(  
  date,  
  title,  
  issuer,  
  url,  
  location,  
  awesomeColors,  
  metadata  
) -> content
```

### **cvPublication**

Add the publications to the CV by reading a bib file.

- bib (bibliography): The bibliography object with the path to the bib file.
- keyList (list): The list of keys to include in the publication list.
- refStyle (str): The reference style of the publication list.
- refFull (bool): Whether to show the full reference or not.

## Parameters

```
cvPublication(  
  bib,  
  keyList,  
  refStyle,  
  refFull  
) -> content
```

## cvSection

Add the title of a section.

NOTE: If the language is non-Latin, the title highlight will not be sliced.

- title (str): The title of the section.
- highlighted (bool): Whether the first n letters will be highlighted in accent color.
- letters (int): The number of first letters of the title to highlight.
- metadata (array): (optional) the metadata read from the TOML file.
- awesomeColors (array): (optional) the awesome colors of the CV.

## Parameters

```
cvSection(  
  title,  
  highlighted,  
  letters,  
  metadata,  
  awesomeColors  
) -> content
```

## cvSkill

Add a skill to the CV.

- type (str): The type of the skill. It is displayed on the left side.
- info (str | content): The information about the skill. It is displayed on the right side. Items can be separated by #hbar().

## Parameters

```
cvSkill(  
  type,  
  info  
) -> content
```

## cvSkillTag

Add a skill tag to the CV.

- skill (str | content): The skill to be displayed.

### Parameters

```
cvSkillTag(skill) -> content
```

### cvSkillWithLevel

Add a skill with a level to the CV.

- type (str): The type of the skill. It is displayed on the left side.
- level (int): The level of the skill. It is displayed in as circles in the middle. The minimum level is 0 and the maximum level is 5.
- info (str | content): The information about the skill. It is displayed on the right side.

### Parameters

```
cvSkillWithLevel(  
    type,  
    level,  
    info  
) -> content
```